Step-by-Step Guide: Building a Standout Coding Project for Your 2026 Job Hunt

Step 1: Identify a Real Problem in a High-Demand Domain

Focus on real-world pain points, not trivial apps. Start by choosing a problem that development teams actually struggle with on a daily basis. Avoid overdone "portfolio projects" like to-do list apps or weather trackers that recruiters have seen a hundred times[1]. Instead, look for issues that affect productivity or quality in modern tech teams. Some high-demand domains in 2026 include:

- **pDeveloper Tools & Productivity:** For example, developers often waste time on code reviews, debugging, writing tests, or documentation. An AI-assisted code review tool or intelligent debugging assistant could alleviate these frustrations. Solving tooling gaps can hugely boost productivity[2], which makes it valuable to employers.
- **DevOps & Infrastructure:** Companies grapple with complex infrastructure and deployments. Ideas here include an anomaly-detection system for logs or an automated incident response bot. Downtime and slow releases are costly, so anything that helps monitor and optimize infrastructure (smart deployment dashboards, risk predictors, etc.) addresses a frequent pain point. Integration headaches between systems are also common[3].
- **Code Quality & Security:** Security bugs and bad code quality can be catastrophic for companies[4]. A project like an AI-powered vulnerability scanner, a code smell detector, or a performance bottleneck analyzer shows you can tackle urgent, high-stakes problems. These areas have clear business impact – preventing breaches or outages is something every company cares about.
- **API & Integration Tools:** Many teams struggle with integrating services and APIs. Consider building a smart API documentation generator, an automated API testing tool, or a schema validation helper. Integrations and compatibility issues are persistent frustrations in development[3], so a tool that makes them easier will stand out as solving a real need.
- **Data Engineering & MLOps:** With the rise of AI, companies need engineers who can manage data pipelines and models in production. A project like a data pipeline monitoring dashboard, a model drift detection tool, or a feature engineering automation framework would hit on contemporary needs. Modern ML/AI systems require monitoring and optimization – portfolios that show MLOps skills prove you're ready for cutting-edge work[5].

Why this matters: Hiring managers in 2026 want to know *"Can this person solve the problems we have right now?"*[6]. By choosing a project in a popular, problem-rich domain (like those above), you immediately signal that you understand the industry's current challenges. In fact, tech recruiters have noted that the best projects are those

with real-world impact and creativity in solving actual problems[7]. So pick a domain/problem where your solution could genuinely save time, reduce errors, or improve efficiency for a team.

Step 2: Research and Validate the Pain Point

Once you've chosen a general problem area, **validate that the pain point is real and significant.** This step is about confirming that companies truly struggle with the issue – you don't want to invest in solving an imaginary or niche problem.

Do user research: Explore developer forums and communities to gauge the pain. Developers often vent about their biggest headaches on Reddit (subreddits like r/webdev, r/devops, r/programming) and Stack Overflow. Look for recurring complaints or highly upvoted threads – those indicate a widespread frustration[8]. For instance, if many threads discuss how tedious writing unit tests is, an "automated test generator" might be a valued project. Also browse issue trackers of open-source projects related to your idea: an issue with dozens of 👍 reactions or a long discussion means the community craves a solution[9].

Learn from industry anecdotes: Tech blogs and engineering postmortems from companies (e.g. Netflix tech blog, Stripe engineering blog) are goldmines for understanding real challenges[10]. If multiple companies have written about the difficulty of scaling a particular system or coping with a type of bug, that validates your project idea. You might discover, for example, that many teams struggle with maintaining API docs – reinforcing the value of an intelligent API documentation tool.

Define your target user and context: As you research, clarify *who* faces this problem and *when*. Is it DevOps engineers triaging incidents at 3 AM? Frontend developers dealing with inconsistent API schemas? Being specific will help you tailor the solution. Moreover, understanding the context (how people currently work around the problem, what the limitations of existing tools are) lets you design a project that truly improves on the status quo. Remember, a strong project addresses a **frequent, intense, and costly** pain point, one that people would gladly have fixed[11][12]. If your research shows the issue is rare or minor, consider pivoting to something more impactful.

Step 3: Define Your Project's Solution and Scope

With a validated problem in mind, **outline a specific solution** that your project will provide. This means writing down what your project will do (and what it won't, at least in the first version). Focus on a clear, concise value proposition – how exactly will it make developers' or companies' lives easier?

Brainstorm features, then trim to a Minimum Viable Product (MVP): It's tempting to think of a dozen cool features, but remember you need to actually build this. List all ideas (for example, if your domain is code quality, features might include scanning code for bugs, suggesting fixes, tracking code coverage, etc.). Then identify the core feature that delivers the most value. **Start with the smallest thing you could build that truly**

solves the core pain[13]. For instance, if your project is an AI code review assistant, the MVP might be a command-line tool that scans a code file for a few common bug patterns and suggests changes. All the extra bells and whistles (nice GUI, multiple language support, integration with GitHub, etc.) can come later. By narrowing scope, you ensure your project is achievable and demonstrates a solid solution to the problem without getting overwhelmed.

Ensure uniqueness or improvement: Ask yourself how your approach is better than or different from existing solutions. It doesn't need to be one-of-a-kind, but it should at least improve on something. Maybe you're using AI in a way others haven't, or you're combining two tools (e.g., log analysis + an alerting Slackbot) in a novel fashion. Clarify this unique angle in your project description. This will not only guide your development but also be a great talking point in interviews ("Most log analyzers do X, but my tool also does Y which addresses _____ gap that many teams complained about.").

Example – Defining a solution: Suppose the pain point is "writing repetitive boilerplate code for APIs is time-consuming." A possible project solution: *a CLI tool that generates API endpoint boilerplate from a specification.* Your MVP feature set might be: given a JSON schema or OpenAPI spec, the tool outputs a ready-to-run code file with all the endpoints and data models. That alone would save developers a lot of time. You might initially target one language/framework (say, Node/Express or Python/FastAPI) to keep scope small. Document this plan clearly before you start coding.

Step 4: Pick the Right Tech Stack and Tools

Choosing the technology stack is crucial – it should be **relevant, modern, and appropriate** for your project. The stack is part of what makes your project impressive, as it shows your familiarity with the tools companies are using in 2026. Here's how to decide:

Match the domain: Use technologies commonly used for that type of problem. For a data engineering or MLOps project, Python is a safe bet (with libraries like pandas, TensorFlow/PyTorch or Docker for pipelines). If you're building a web-based developer tool or dashboard, consider JavaScript/TypeScript with a popular framework (e.g. a Next.js or React front-end, Node.js or Django backend). DevOps-related project? You might incorporate infrastructure-as-code tools (Terraform, Kubernetes), or at least Docker containers. Choosing a familiar stack for the domain reassures recruiters that you'd be effective in their environment. (For example, an API testing tool built with Node.js and Express shows you know the same ecosystem their web team likely uses.)

Incorporate trending tech: 2025 and 2026 are seeing big shifts with AI and other cutting-edge tech becoming standard. If it fits your project, leveraging AI/ML will make it especially eye-catching. For instance, use an OpenAI API or a local LLM model if you're doing an "intelligent" code assistant or log analyzer – showing you can work with AI APIs and prompt engineering (a very hot skill now). Or if your project is about infrastructure optimization, maybe incorporate a small machine learning model that

learns usage patterns. Recruiters have noted a surge in AI/ML roles and expect portfolios to reflect that shift, yet many candidates still show outdated projects[14]. By working with the **technologies companies are "betting on" for the future (think LLMs, cloud services, edge computing)**, you signal that you'll hit the ground running in a modern tech team.

Balance difficulty with familiarity: If you're a new engineer, it's okay if not every part of the stack is something you've mastered before – a project is a learning experience. But do ensure you have enough baseline knowledge or resources to realistically use the tools you pick. It's better to build a solid project in Python/JavaScript that works, than to half-implement something in a very complex new language just for hype. That said, learning a bit of a new tech can also be a bonus – e.g. using **Docker** to containerize your app or setting up a **GitHub Actions** CI workflow (even if you've never done that before) will greatly impress recruiters with your initiative in DevOps. Just allocate time to learning and keep the core of your project in a language you're comfortable with so you don't stall.

Example – Tech stack selection: For our hypothetical *AI code review assistant CLI*, we might choose Python (since it has good ML libraries and many static analysis tools) and use the OpenAI API for natural language suggestions. We could also use a library like AST parsing for code or integrate with Git hooks (to run the tool automatically on commits). If aiming to show extra DevOps, we might plan to containerize it with Docker so it's easy to run anywhere. This combination (Python + AI API + Docker) is very relevant to industry trends and demonstrates both software and DevOps skills.

Step 5: Develop the Project in Iterative Steps

Now, roll up your sleeves and start **coding**! Treat your project like a mini-software endeavor with good practices. Here's a game plan for development:

Break it into tasks: Divide the work into smaller pieces or milestones. For instance, Milestone 1: set up the basic project structure and a "Hello World" for your tool. Milestone 2: implement the core functionality (e.g., the log parsing engine or the code scanning logic). Milestone 3: integrate the AI or advanced feature (if any). Milestone 4: add a simple UI or CLI interface. By breaking it down, you can make consistent progress and test as you go.

Build the MVP first: Stick to the minimum feature set you decided on. Get that working end-to-end before adding extras. For example, ensure your automated test generator can read a sample code file and output one test file correctly, *before* you try to add multi-language support or a fancy UI. A functional MVP, even if small, is far more valuable than a half-finished ambitious project. Plus, you can always say, "here's V1, and here are ideas I'd add if I had more time" – showing awareness of how to iterate.

Apply best practices: Even though this isn't a production app, treating it professionally will make it stand out. Use version control (create a Git repository from the start and commit changes logically with clear messages). Write clean, readable code with

comments where needed. If applicable, include unit tests for critical parts of your code (this shows you care about quality – a big plus if your project *is* about code quality or testing!). Manage your dependencies thoughtfully (e.g., use a `requirements.txt` or package.json). These details demonstrate craftsmanship. In fact, recruiters scanning GitHub will notice things like a well-structured repository and testing approach as signs of a mature engineer[15][16].

Stay user-focused: Keep referring to the problem and user you defined. As you implement, ask "does this solve the user's pain point in a simple way?" It's easy to get lost in technical rabbit holes or gold-plating features. For instance, if building a devops monitor, you might be tempted to perfect a complex UI, but if the core alerting doesn't work reliably, the project fails its purpose. Prioritize functionality that addresses the pain. Every time you make a design choice, think about how it serves the end goal of easing that real-world problem.

Step 6: Add a "Wow" Factor (Enhance and Innovate)

Once the basic solution is working, **consider what extra features or improvements** could make your project truly shine. This is your chance to differentiate your project and show creativity and initiative beyond the bare minimum. A few ways to add a wow factor:

- **Integrate AI smartly:** If you haven't already, see if an AI component could enhance your project. For example, your initial log analyzer might just regex through logs for errors – could you train or use an ML model to detect anomalies or predict incidents? Or if you built a security scanner, maybe incorporate an AI that explains the vulnerabilities it finds in plain English for the user. Use AI where it genuinely improves the result, showing you know how to leverage modern tools. (Keep in mind computational limits and be ready to explain why you chose a certain model or API – that insight impresses interviewers.)

- **Polish the user experience:** Little UX touches can make your project feel professional. If it's a CLI tool, maybe add colorized output or a nicely formatted summary report. If it's a web app, invest some time in a clean UI with responsive design. You could also add convenience features, like saving user preferences or providing progress indicators during long tasks. A smooth UX shows empathy for users – something employers value in engineers, not just raw coding ability.

- **Ensure performance and scalability (if relevant):** Did you notice your app runs slow on large inputs? Optimize it. Maybe add caching to your AI code reviewer so it doesn't recompute on unchanged files (like one developer did to cut reviews from 10 minutes to 30 seconds)[17]. Or if your data tool processes streams, ensure it can handle a realistic volume. Then you can brag that your design handles scale (and mention how you'd further scale it in the future). This level of thinking is rare in junior projects and will set you apart.

- **Gather feedback and iterate:** If possible, have a friend or colleague act as a beta user. Watch them use your tool (or have them read your README and see if it makes sense). Their feedback might highlight an area to refine – maybe an error message that was unclear or a missing piece of documentation. Iterating based on real feedback, even minor tweaks, shows that you care about usability and quality. It mimics the real-world development cycle of releasing and refining.

In this step, you're effectively **turning a good project into a great project**. The enhancements don't have to be huge or numerous – even one well-executed innovative feature can make a recruiter or hiring manager say "oh, that's clever!" and remember your work. It demonstrates that you don't just do the bare minimum; you go above and beyond, which is a trait of a high-value engineer.

Step 7: Test Thoroughly with Realistic Scenarios

Now that your project has all its key features, make sure it **actually delivers on the promise** in a realistic context. Testing isn't just about unit tests (though if you wrote those, great) – it's also about using the project as if you were the target user and seeing how it holds up.

Use real or realistic data: If your project is a log analysis tool, feed it large, messy log files (you can find open-source log datasets or generate synthetic ones). Does it still perform well? Does it find the kind of anomalies you expect? If it's an API tool, try it on a real API spec or live API endpoints. By testing with real-world inputs, you might uncover edge cases or performance issues. This also gives you concrete results to talk about. For example, you could say, "I ran my vulnerability scanner on an open-source project and it identified 3 critical issues that the maintainers later confirmed." Such anecdotes are impressive in interviews.

Simulate the user journey: Run through the typical usage of your project step by step. If it's a web app, from sign-up to final output; if it's a CLI, from installation to executing commands. Note any rough edges and fix them. Maybe the installation was complicated – you might containerize the app or improve the setup script. Perhaps the output was hard to interpret – you add a summary table or visualization. Ensure the project is **usable and reliable** in the hands of someone who isn't its developer (imagine a stranger trying it based on your instructions).

Measure impact if possible: This is a pro tip – if you can quantify how well your project performs, do it. Did your developer tool cut down the time to do X by 50% compared to the manual way? Does your AI assistant fix 80% of common errors it scans? Any metric or result that demonstrates effectiveness is gold. It ties your project to a *business outcome or real improvement*, which is exactly what impresses employers[18]. For instance, *"My CI/CD pipeline setup reduced deployment time from 30 minutes to 5 minutes by automating tests and deployments."* That kind of statement in your resume or interview shows you understand the **value** of your work, not just the technical steps. Even if you can't get precise numbers, qualitatively describing the improvement (like

“more secure”, “faster troubleshooting”, “better developer experience”) with evidence from testing will strengthen your case.

Finally, fix any bugs you encountered during testing. Polish the edges – this might be the last coding you do on the project, so make it count. The goal is to have a project that not only runs without crashing, but truly solves the problem you set out to solve, in a way that could be dropped into a workplace and start helping immediately. As one portfolio guide put it, the strongest projects *“feel like they could be dropped into production tomorrow”*[19] – that’s the vibe you’re aiming for.

Step 8: Document and Package Your Project Professionally

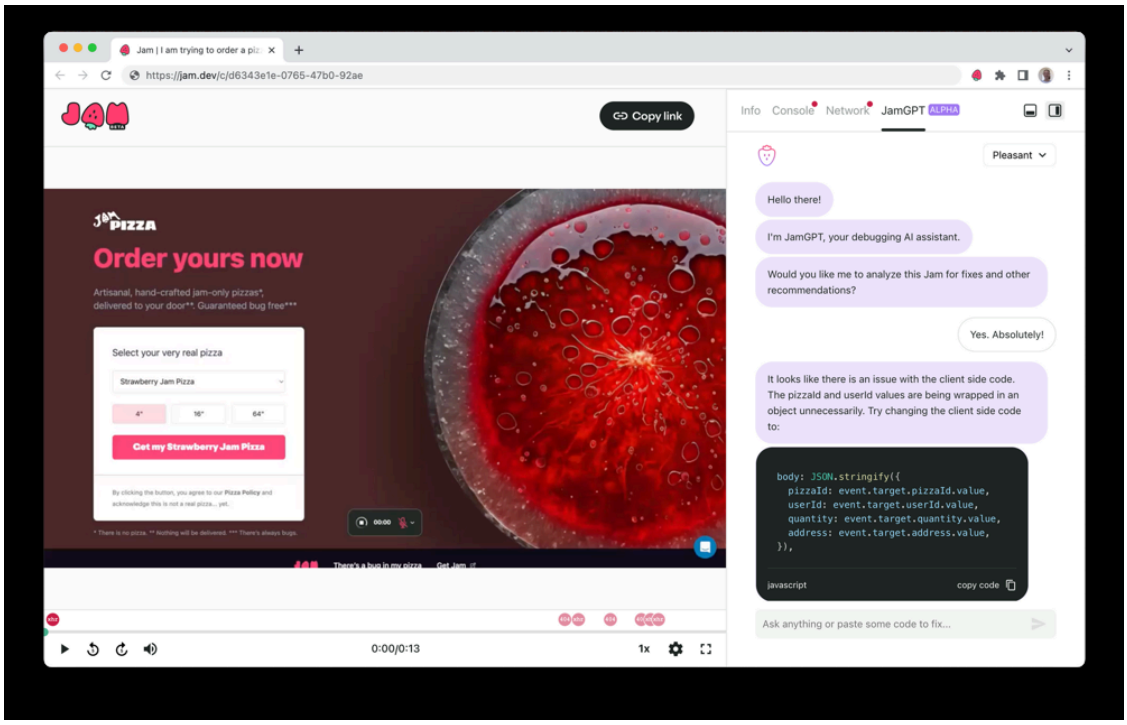
With a working project in hand, don’t neglect documentation and presentation – these often make the difference in getting your project noticed. A well-documented project shows professionalism and helps others (like recruiters or interviewers) understand your work quickly. Here’s what to do:

Write a great README: Your repository’s README.md is essentially the landing page for your project. Make it clear and informative. At minimum, include: a brief description of the problem and your solution, a list of features, instructions on how to install/run the project, and an example of it in action. Start with a strong pitch of the project: e.g., “**AI LogSense** – An AI-powered log analyzer that detects anomalies in server logs and alerts developers to potential issues *before* downtime strikes.” Then explain usage. A good README might also include an architecture diagram or workflow diagram to show how your system works internally[20] – this is a nice touch that conveys the complexity and design of your project at a glance. If you have results from Step 7 (like performance metrics or screenshots of output), include those too!

Comment and clean your code: Go through your codebase and make sure it’s tidy. Remove debug print statements, refactor any extremely messy parts, and comment any non-obvious logic. You want someone browsing your code to think, “This person writes clean code.” Recruiters might not read every line, but they might scan a few files or note the folder structure. If you claimed to build a code quality tool but your own code is disorganized, that’s not a good look. Show that you can apply best practices to your own work (linters, consistent formatting, etc.). Also, ensure you include any configuration or environment files needed to run the project (like a requirements.txt, package.json, or Dockerfile). A strong project is **reproducible** – someone else should be able to get it running by following your instructions[15][16].

Add visuals: Wherever possible, use visuals to make your project engaging. This could be screenshots (if it has a UI), terminal output captures (if CLI), or even a short demo GIF/video. For example, include an image of your tool’s dashboard or a snapshot of your AI assistant fixing a bug in code. Visuals draw the eye and can often communicate the essence of your project faster than text. At the very least, consider adding one at the top of your README to spark interest. *(Pro tip: if you deploy a web app, also add a few*

screenshots of it in the README, so even if the link is clicked by a recruiter after it's offline, they see what it looked like.)



Example: An AI debugging assistant in action. Tools like this integrate into developers' workflow to automatically identify and suggest fixes for bugs, saving time and reducing frustration. Including a screenshot of your project's interface or output can quickly show its purpose and impact.

Create user-friendly touches: If appropriate, package your project for easy use. For instance, publish it as an NPM package or PyPI package if it's a library/CLI tool, so others can install with one command. Or provide a Docker image. This isn't required, but it demonstrates extra effort to make your project accessible and signals engineering maturity. Even providing a one-line script to set up prerequisites can help.

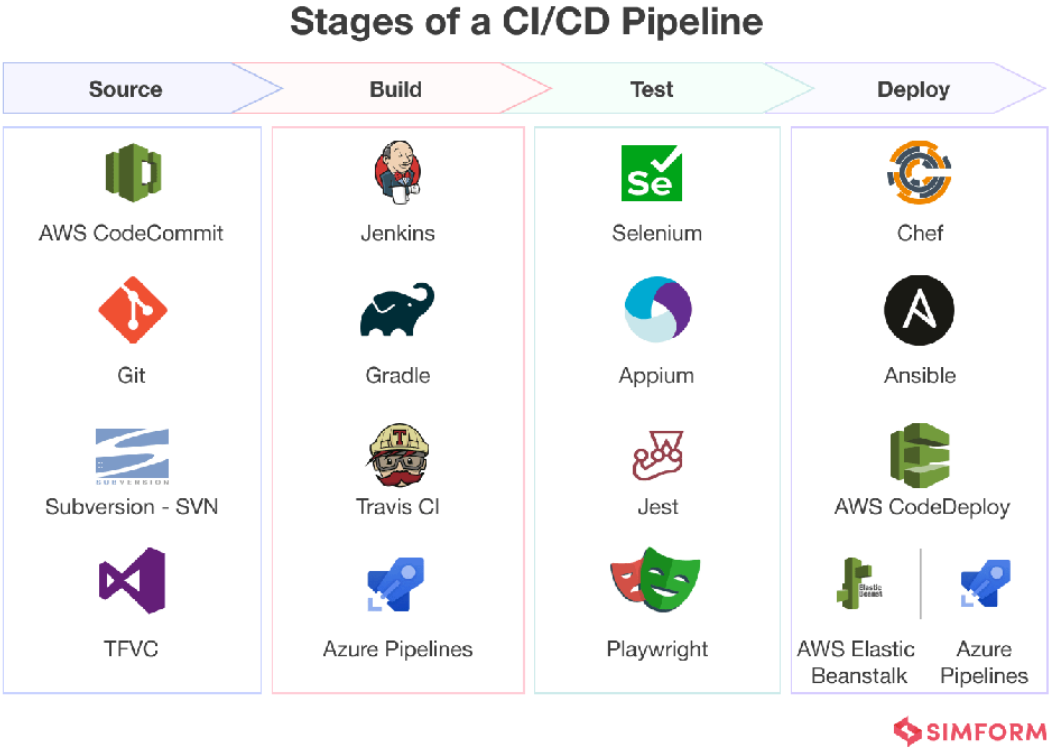
Before moving on, double-check your writing for clarity and typos – communication counts. A hiring manager reading your project documentation is also indirectly evaluating your communication skills. With thorough documentation, you're telling the reviewer *exactly* what your project is and why it matters within seconds, which is critical given how little time they might spend perusing your portfolio[6]. All these documentation steps ensure your hard work in coding doesn't go unnoticed due to a lack of clarity.

Step 9: Deploy and Showcase Your Work

A project that's live or easy to demo will **immediately set you apart**. It shows end-to-end capability and gives recruiters a chance to experience your work hands-on. Here's how to put your project out there and maximize its visibility:

Deploy it (if feasible): For web applications or anything with a UI, deploy it to a free hosting service if you can. Platforms like Vercel, Netlify, or Render can host your web apps with minimal fuss, and cloud providers or Heroku can host backends/databases on free tiers. A deployed project means a recruiter can click a link and see it in action *right away*, which is powerful. If it's not a web app (say, a CLI tool or library), consider providing a live demo in another form – for example, a YouTube video walkthrough, or an interactive notebook if it's data-related. The key is to reduce friction for someone to appreciate what you built. In your README and resume, include the live link or demo link prominently (and label it "Live Demo").

Set up a CI/CD pipeline: This is an optional but highly recommended step for deployment and polish. Configure continuous integration and deployment for your project – for instance, using GitHub Actions to run tests and automatically deploy the latest version to your host. Setting up a CI/CD pipeline not only proves your DevOps savvy, it also keeps your project running smoothly for viewers. Recruiters absolutely love seeing that a candidate can *"ship code that's production-ready"*, not just write code[21]. By containerizing your app (e.g., with Docker) and using CI/CD, you demonstrate engineering maturity. Consider the typical stages of a pipeline (source -> build -> test -> deploy) and try to implement a basic version of it for your project



. Even a simple automated deployment on push is great. You can mention in your documentation that you did this ("CI/CD pipeline set up with automated tests and deployment to XYZ – see badge in repo"), which is sure to impress.

For example, if you built an API tool, you might show that every push runs your test suite and then deploys the latest API to a cloud function. This level of completeness mirrors real-world software practices and will be a highlight of your project.

Promote your project in the community: Write a short blog post or article about your project and the problem it solves. Platforms like Medium, Dev.to, or even a LinkedIn article can be good outlets. In the blog, discuss why you built it, how you built it, and any interesting findings or challenges. This not only solidifies your understanding (good for interviews) but also demonstrates communication skills. Plus, you might catch the eye of someone online who finds it cool – you could get users, feedback, or even a referral. You don't need to be an expert writer; just share your genuine experience building the project. As one guide suggests, *"write a case study explaining what you built"* as part of showcasing your project[22].

Share and network: Tweet about it (or post on X), share on LinkedIn (especially mentioning you built this project to solve a real problem – recruiters on LinkedIn may notice), and post in relevant communities (for example, the r/SideProject or r/learnprogramming subreddit, or niche communities related to your project domain). The goal is to show that you're an active, passionate engineer. You never know – a well-received project can sometimes go viral or at least get you some kudos from professionals in the field. Even if it doesn't, you're creating a digital footprint of being a builder and problem-solver, which can only help your job search.

Lastly, ensure your resume and portfolio site (if you have one) prominently feature this project. Add a bullet point in your resume under a "Projects" section, describing it in one line *with an impact metric if possible*. For example: *"Built an AI-powered resume reviewer web app – used by 250+ people to get instant feedback on CVs, improving their interview call rates[23]."* This kind of description (problem + solution + outcome) will pique interest. By deploying and showcasing your project effectively, you convert all your hard development work into a tangible asset that employers can see and appreciate.

Step 10: Leverage Your Project in Job Applications and Interviews

Your project is now out in the world – the final step is to **make it work for you in landing that job**. This means highlighting it strategically and being prepared to discuss it with enthusiasm and depth.

Tailor your resume/portfolio highlights to the job: For each application, consider how your project aligns with the company or role. If you're applying to a DevOps role and your project was a CI/CD pipeline tool, emphasize the DevOps aspects: mention the tools (Docker, AWS, etc.) you used and the deployment pipeline you set up. If the role is more software engineering general, focus on how your project shows end-to-end development and problem-solving. Always include a direct link to the code (GitHub) and live demo in your resume or online portfolio – make it easy for them to check it out. Many recruiters *will* glance at your GitHub, but they might only spend **seconds** there[6],

so ensure the project you want them to see is front and center (pin it on GitHub if needed). One LinkedIn tech recruiter advised: *"If a company can see how your project could be useful, you're already ahead."*[24] So, frame it in terms of usefulness to the company's domain if you can.

Prepare your "project story": In interviews, almost inevitably, you'll be asked about your project (because it's interesting and relevant!). Be ready to talk about it clearly and enthusiastically. Structure your explanation like this: the problem you identified, why it's important (perhaps mention how frequent/costly it is for teams – you have that research in your pocket), the solution you built, challenges you faced while building it, and the outcome (what it achieves now, any results or feedback). This mirrors the STAR method (Situation, Task, Action, Result) often used in behavioral interviews, but applied to your project. For example: *"I noticed many developers struggle with flaky tests that waste time (situation). I decided to tackle this by building an AI tool that automatically suggests fixes for flaky tests (task/solution). I encountered challenges with false positives and had to tweak the ML model extensively (challenge/action), but in the end it successfully reduced flaky test occurrences by 30% in my trials (result)."* Such a narrative shows your problem-solving mindset.

Demonstrate learnings and passion: Companies love to see that you learned something from a project and that you're genuinely passionate about tech. Don't shy from discussing parts that were tough or new to you – maybe you struggled to set up a Kubernetes cluster for deployment and it failed, but you learned Docker as a fallback. That shows resilience and learning ability. Also, be ready to dive into technical details if asked (since you wrote it, you should know it inside out). For instance, an interviewer might ask, "How did you implement the anomaly detection logic?" – you can then explain the algorithm or model you used. This is your chance to shine as someone who not only had a cool idea but also can communicate the technical intricacies clearly.

Connect it to the company's needs: If you have insight into what the company does, draw parallels. *"Your job description mentions migrating to microservices – interestingly, my project involved building a containerized CI/CD pipeline, so I'm familiar with the challenges of deploying many services continuously."* Making that connection explicitly can leave a strong impression. It tells the interviewer that you *get* their problems and have even preemptively worked on similar ones.

Finally, convey your enthusiasm. This project is something you chose to build – let that positive energy show. Employers appreciate genuine curiosity and drive. You solved a real-world problem on your own; that speaks volumes about your initiative. In fact, many hiring managers consider such projects as evidence that you can **"solve real problems, not just write code"**[25]. By walking them through how you identified and solved the problem, you're essentially doing the job in front of them.

In summary, this step-by-step journey – from picking a relevant pain point, building a targeted solution, and showcasing it professionally – will greatly increase your chances of getting noticed and hired. You'll have demonstrated technical skills, problem-solving,

user empathy, and the ability to drive a project to completion. In 2026's competitive tech job market, a project like this isn't just a portfolio piece; it's proof that you can deliver value. Good luck, and happy building!

Sources:

- Recruiter insights on portfolio projects[1][7]
- Identifying real developer pain points[8][9]
- Common technical challenge domains in industry[3][4]
- Advice on scoping and MVP focus[13]
- Emphasizing production-ready and end-to-end skills[21][20]
- Highlighting project impact and results[23][18]

---

[1] [7] [24] [25] . | Sajjaad Khader

https://www.linkedin.com/posts/sajjaad-khader_%F0%9D%97%9C-%F0%9D%97%AE%F0%9D%98%80%F0%9D%97%B8%F0%9D%97%B2%F0%9D%97%B1-%F0%9D%97%AF%F0%9D%97%B6%F0%9D%97%B4-%F0%9D%98%81%F0%9D%97%B2%F0%9D%97%B0%F0%9D%97%B5-%F0%9D%97%BF%F0%9D%97%B2%F0%9D%97%B0%F0%9D%97%BF%F0%9D%98%82%F0%9D%97%B6%F0%9D%98%81%F0%9D%97%B2%F0%9D%97%BF%F0%9D%98%80-activity-7300922704222711809-QBjG

[2] [3] [4] [8] [9] [10] [11] [12] [13] Technical Pain Points: How to Identify and Solve Them in 2026 - PainOnSocial Blog | PainOnSocial

https://painonsocial.com/blog/technical-pain-points

[5] [15] [16] [18] [19] [20] ML Engineer Portfolio Projects That Will Get You Hired in 2025 - Interview Node Blog

http://www.interviewnode.com/post/ml-engineer-portfolio-projects-that-will-get-you-hired-in-2025

[6] [14] Portfolio Roadmap 2026: 5 Projects That Get Interviews | by The Shipping Engineer | Medium

https://medium.com/@ashusk_1790/portfolio-roadmap-2026-5-projects-that-get-interviews-ddcb9716b46b

[17] Building an AI Code Reviewer in 2 Days | by Rachel Cantor | Medium

https://medium.com/@rachelcantor/building-an-ai-code-reviewer-in-2-days-49c6bcad8ed8

[21] [22] [23] 10 Genius Technical Projects That Can 10x Your Resume in 2025 - DEV Community

https://dev.to/finalroundai/10-genius-technical-projects-that-can-10x-your-resume-in-2025-454c